

Faculty of Computer Technology and Cybersecurity
Department of Computer Engineering
ARCHITECTURE AND ORGANIZATION OF COMPUTER SYSTEMS
LABORATORY WORK - 3

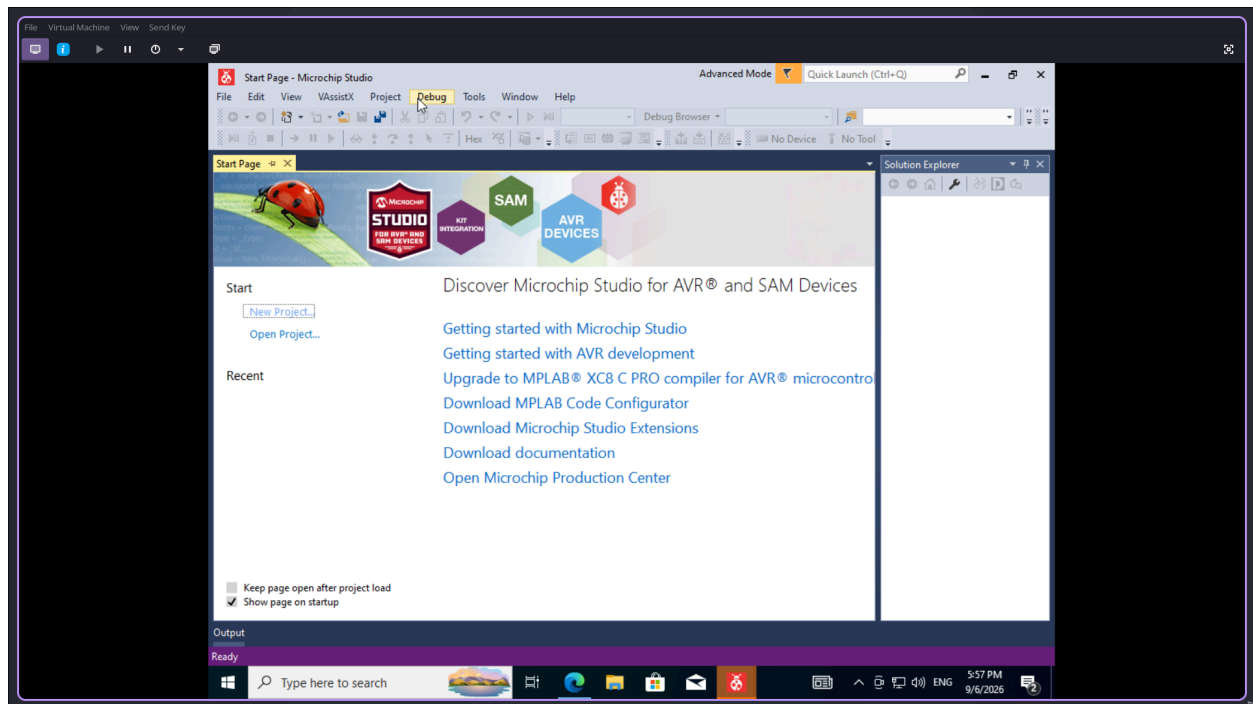
Date: 26.09.2026

Full Name: Daniil Kalts

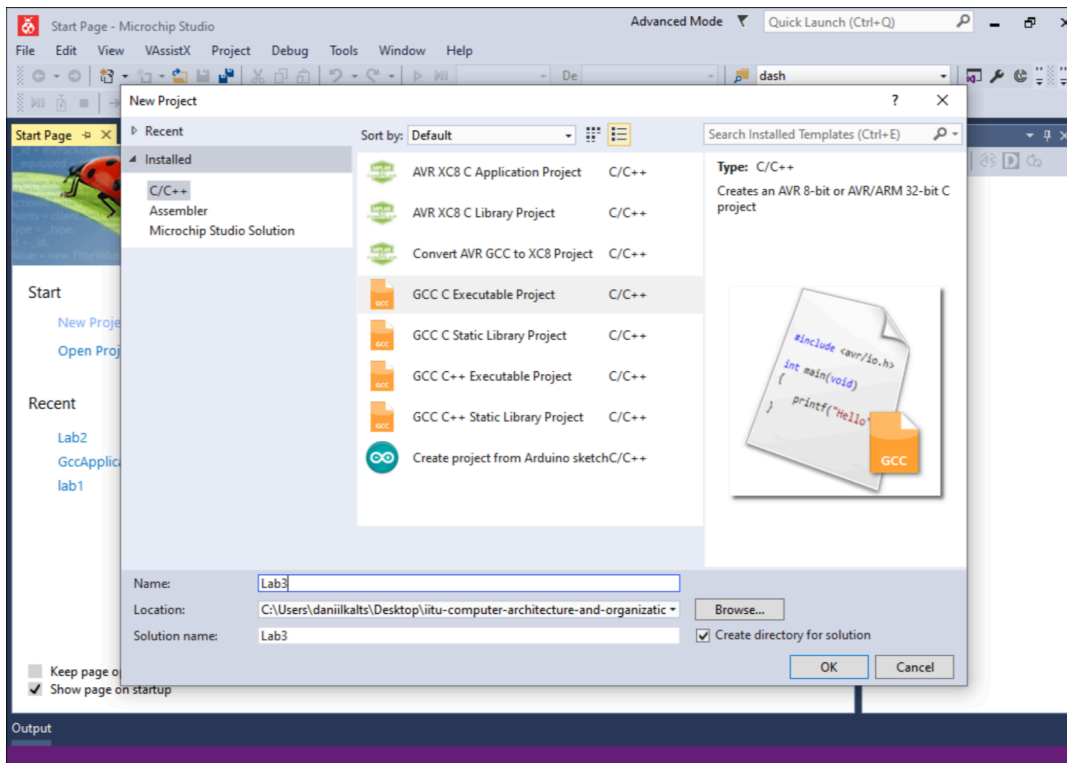
Group: IT2-2404SE

PROGRESS OF WORK

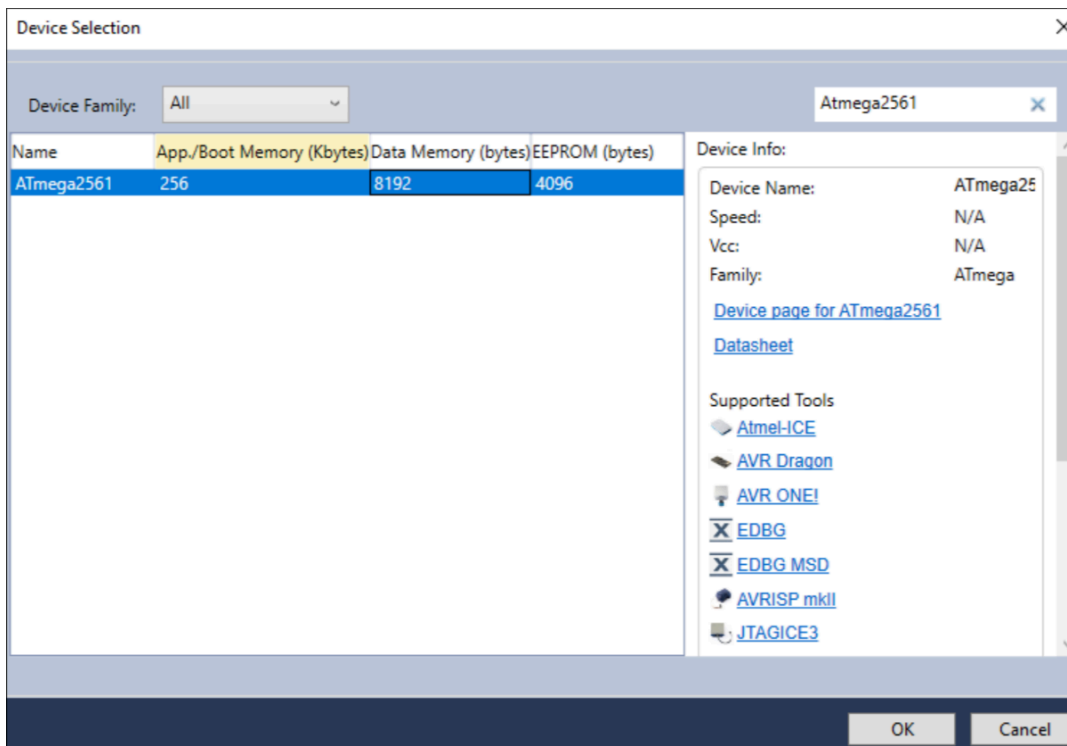
1) Start the Microchip Studio.



2) Create a new GCC C Executable project.



3) Select the device (in this work — ATmega2561).



4) Next, program the microcontroller.

```
#define F_CPU 8000000UL

#include <avr/io.h>
#include <util/delay.h>

void pwm_init(void)
{
    DDRB |= (1 << PB7);

    TCCR0A = (1 << WGM00) | (1 << WGM01) | (1 << COM0A1);
    TCCR0B = (1 << CS00);

    OCR0A = 0;
}

int main(void)
{
    pwm_init();

    DDRA &= ~(1 << PA0);
    PORTA |= (1 << PA0);

    int brightness = 0;
    int direction = 1;

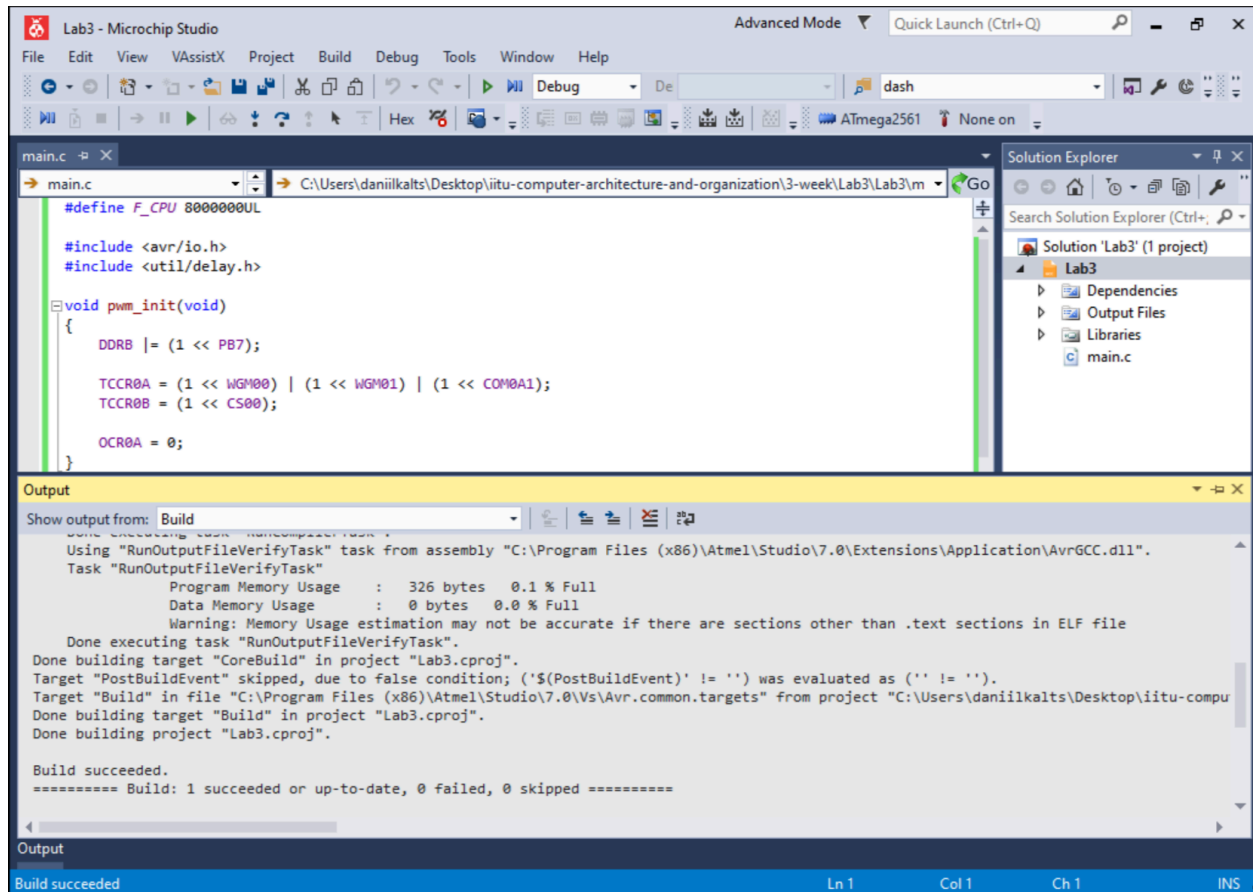
    while (1)
    {
        if (!(PINA & (1 << PA0)))
        {
            brightness += direction;

            if (brightness >= 255)
            {
                brightness = 255;
                direction = -1;
            }
            else if (brightness <= 0)
            {
                brightness = 0;
                direction = 1;
            }

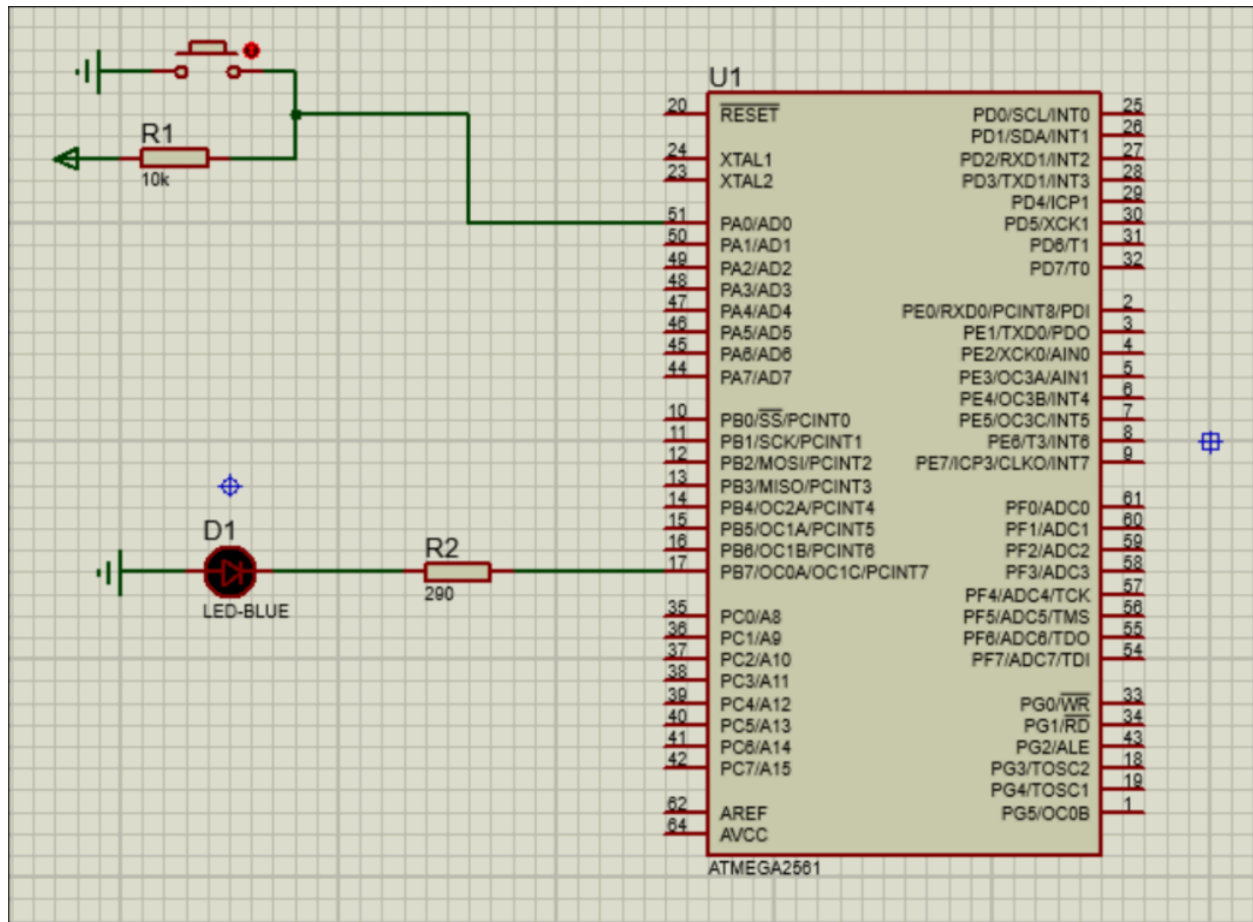
            OCR0A = brightness;
        }

        _delay_ms(10);
    }
}
```

5. Compile the project and obtain the .hex file.



6. Open Proteus (ISIS Professional) and pick the ATmega2561, Button, LED, Resistors, Power and Ground.



7. Load the compiled .hex file into the microcontroller's properties (Program File) and configure the component

Edit Component

Part Reference: Hidden: ☐

Part Value: Hidden: ☐

Element:

PCB Package:

Program File:

CLKDIV8 (Divide clock by 8)

CKOUT (Clock output)

WDTON (Watchdog timer always on)

BOOTRST (Select Reset Vector)

CKSEL Fuses:

Boot Loader Size:

SUT Fuses:

Advanced Properties:

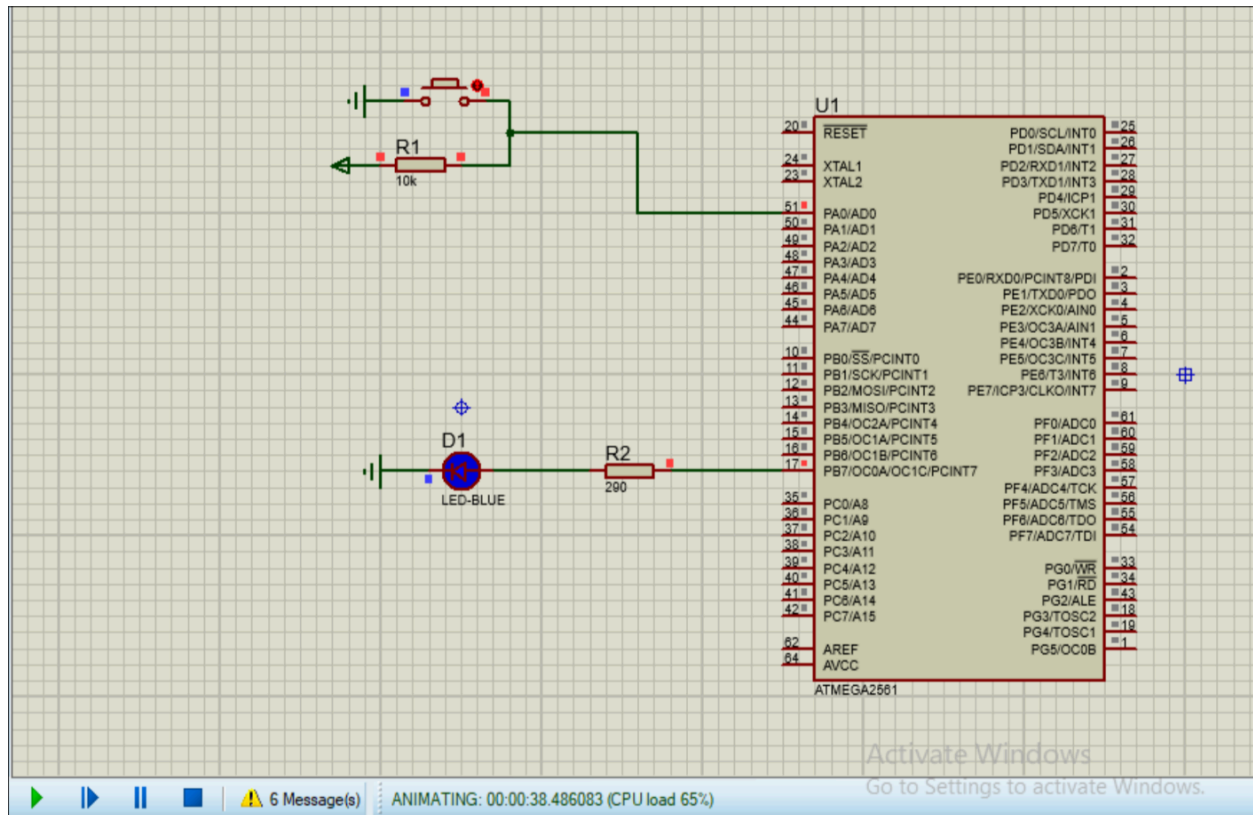
Other Properties:

☐ Exclude from Simulation ☐ Attach hierarchy module

☐ Exclude from PCB Layout ☐ Hide common pins

☐ Exclude from Current Variant ☐ Edit all properties as text

8. Run the simulation and verify that the LED smoothly brightens and dims while the button is held, then stays at its current brightness when the button is released.



Conclusion:

In this laboratory work, I programmed the ATmega2561 microcontroller in C and tested the circuit in Proteus. Timer0 generated a PWM signal on PB7 to control the LED brightness, while a button on PA0 controlled the fading. When I held the button, the LED smoothly brightened and dimmed; when I released it, the brightness stayed at its current level.